

Guide d'Utilisation : Pipeline de Collecte de Données

1. Préparation de l'environnement technique

Avant d'exécuter le pipeline de collecte, plusieurs outils doivent être installés et configurés.

1.1 Installation de Python

Le projet a été développé avec Python 3.

Installation recommandée :

- Python 3.11 ou supérieur ;
- environnement virtuel (**venv**) conseillé.

Création de l'environnement virtuel :

```
python -m venv venv
```

Activation de l'environnement :

Windows

```
venv\Scripts\activate
```

Linux / macOS

```
source venv/bin/activate
```

1.2 Installation des dépendances Python

Les bibliothèques nécessaires doivent ensuite être installées.

Commande d'installation :

```
pip install requests python-dotenv langdetect psycpg2
```

Bibliothèques utilisées

Bibliothèque	Rôle
requests	Communication avec l'API Bluesky
python-dotenv	Gestion des variables d'environnement
langdetect	Détection automatique de langue
psycopg2	Connexion à PostgreSQL
re	Nettoyage des textes
datetime	Gestion des dates

2. Installation et configuration de PostgreSQL

2.1 Installation de PostgreSQL

Le projet utilise PostgreSQL pour stocker les données collectées.

Après installation :

- démarrer le serveur PostgreSQL ;
- créer un utilisateur si nécessaire ;
- définir un mot de passe administrateur.

2.2 Création de la base de données

Une base dédiée au projet doit être créée.

Commande SQL :

```
CREATE DATABASE fake_news_project;
```

Une fois créée, la base servira à stocker :

- les données brutes collectées ;
- les données nettoyées ;
- les futurs résultats d'analyse NLP.

2.3 Configuration des variables d'environnement

Un fichier `.env` doit être créé à la racine du projet.

Exemple :

```
BLUESKY_HANDLE=votre_identifiant  
BLUESKY_PASSWORD=votre_mot_de_passe  
  
POSTGRES_HOST=localhost  
POSTGRES_PORT=5432  
POSTGRES_DB=fake_news_project  
POSTGRES_USER=votre_nom_utilisateur  
POSTGRES_PASSWORD=votre_mot_de_passe
```

Cette méthode permet :

- d'éviter de stocker les identifiants dans le code ;
- de sécuriser les accès ;
- de faciliter le déploiement.

3. Exécution du Projet

Le projet peut être exécuté de deux manières distinctes, offrant une flexibilité entre une interface graphique simplifiée (Streamlit) et une exécution manuelle via le terminal (ligne de commande).

3.1 Méthode 1 : Exécution via Streamlit (Interface Graphique)

Cette approche offre une interface utilisateur intuitive pour l'ensemble du pipeline.

Pour s'assurer de l'installation de toutes les dépendances on fait dans le terminal : `pip install streamlit pandas plotly psycopg2-binary python-dotenv requests langdetect torch transformers tqdm codecarbon numpy`

Lancer streamlit : `streamlit run app_streamlit.py`

Puis ouvrir sur un navigateur : `http://localhost:8501`

L'utilisateur interagit avec des boutons pour déclencher les étapes (dans l'ordre) :

- Collecte des données
- ETL & Nettoyage
- Classification Fake News
- Analyse émotionnelle

- Fake News x Émotions
- Green IT

3.2 Méthode 2 : Exécution Manuelle via Terminal (Ligne de Commande) sans la partie Green IT

Cette méthode permet un contrôle plus granulaire en exécutant chaque étape via des scripts Python, ce qui est particulièrement adapté aux environnements sans interface graphique.

3.2.1 Architecture générale de la pipeline

La solution repose sur une architecture ETL (*Extract – Transform – Load*).

Nous avons deux scripts principaux :

Script	Fonction
collect.py	Collecte et stockage brut
etl_bluesky.py	Nettoyage et transformation

Cette séparation permet :

- une meilleure maintenance ;
- une architecture plus scalable ;
- une réutilisation plus simple des données.

3.2.2 Collecte et Traitement des Données via Bluesky

Le script `collect.py` permet d'automatiser la récupération des publications depuis Bluesky. Les publications collectées sont stockées dans une table nommée `raw_posts`.

Le nettoyage permet de rendre les textes exploitables pour les futurs traitements NLP. Cette étape est réalisée par le script `etl_bluesky.py`. Une fois nettoyées, les données sont stockées dans une seconde table nommée `processed_posts`.

3.2.3 Lancement du Pipeline ETL

3.2.3.1 Lancement de la collecte

Une fois PostgreSQL configuré et le fichier `.env` renseigné, la collecte peut être exécutée.

Commande :

```
python collect.py
```

Le script :

- se connecte à Bluesky ;
- récupère les publications ;
- filtre les langues ;
- stocke les données brutes.

3.2.3.2 Lancement de l'ETL

Après la collecte, le pipeline de nettoyage peut être lancé.

Commande :

```
python etl_bluesky.py
```

3.2.3.3 Lancement de la classification

Après le nettoyage des données, l'étape de classification (détection de fake news) peut être lancée.

Commande :

```
python classification.py
```

3.2.3.4 Analyse des émotions

Enfin, l'analyse émotionnelle des publications peut être effectuée.

Commande :

```
python emotion.py
```